

BRANDON DINH

📞 714-262-7877 ✉ brandondinhm9@gmail.com Orange County, California

🌐 gitmonkey00.github.io/Brandon-Dinh-Portfolio/ 🌐 <https://linkedin.com/in/brandontdinh>

Education

California State University, Long Beach

Long Beach, CA

Bachelor of Science in Computer Engineering Technology

Experience

DrKumo Inc. – QA & Automation Engineer

April 2024 – Current

- Developed automated testbenches to verify functionality and reliability of websites, authentication flow, and backend response tracking using unittest, Selenium, and custom Python scripts.
- Reduced regression failures from 75 down to 3 by isolating defects from automation noise, improving overall reliability by 96% from resolving failed UI automation flows by improving synchronization logic, and stabilizing brittle test paths.
- Integrated automated tests into CI/CD (Jenkins) pipelines for regression runs, ensuring continuous release monitoring.
- Designed Python automation scripts to generate structured reports from patient data pipelines, exporting to Excel/CSV to visualize and isolate bugs from automated web testbench runs and accelerate end-to-end debugging.
- Collaborated with developers to discover backend issues through automation, contributing to faster defect turnaround.

Certifications

SystemVerilog for Verification - v22.18 | Siemens Xcelerator Academy

July 2025

- Developed reusable, object-oriented UVM environments using class inheritance and composition for FPGA testbenches.
- Connected virtual interfaces to DUT ports to drive and monitor signal transactions across agents, drivers, and monitors.
- Applied constrained randomization and coverage-driven verification using cover groups, cover points, and cross coverage to achieve functional coverage for digital logic. Used weighted and transition bins to prioritize corner-case conditions.
- Implemented parallel verification processes using fork-join, join_any, and join_none in the top-level module to synchronize threads, control DUT stimulus, coordinate driver, monitor, and scoreboard behavior for testbench execution.
- Integrated scoreboards, FIFOs, and reference models for self-checking validation of expected vs. actual DUT responses.

SystemVerilog UVM Certification | Siemens Xcelerator Academy

In Progress

- Currently completing modules on UVM transactions, drivers, monitors, scoreboards, and coverage collectors.
- Building foundational skills in object-oriented testbench design, virtual interfaces, and functional coverage.
- Progressing through 9 chapters including UVM Overview, Sequences, Agents, Factory, and Complex Tests.

Projects (Click 🔗 to View)

AXI4-Lite Bus Protocol UVM Verification for Embedded SoC 🔗 | SystemVerilog, UVM, AXI4-Lite

- Developed a complete UVM-based verification environment for an avionics-style Environmental Sensor Interface Unit (ESIU) with an AXI4-Lite register map for temperature, pressure, configuration, and status monitoring.
- Implemented reusable UVM components (driver, sequencer, monitor, agent, scoreboard) and created constrained-random sequences to validate register behavior, reset logic, illegal accesses, and back-to-back AXI4-Lite transactions.
- Verified AXI4-Lite protocol using SystemVerilog Assertions (SVA), ensuring correct VALID/READY handshakes, address alignment, write responses, and read-data timing while detecting mismatches through a functional scoreboard.
- Designed functional coverage models and performed fault-injection testing (out-of-range sensor values, FIFO overflow, malformed transactions) to validate robust error handling and corner-case behavior.

SystemVerilog ALU Design and UVM Verification with RISC-V ISA 🔗 | SystemVerilog, UVM, RTL

- Designed a SystemVerilog Arithmetic Logic Unit (ALU) implementing a RISC-V-style RV32I instruction set architecture (ISA) including ADD, SUB, AND, OR, XOR, and SLT similar to ALU operations used in modern RISC-V decoders.
- Programmed a SystemVerilog UVM testbench to verify operations and edge cases, such as zero, overflow, and signed arithmetic in digital logic for functional verification.
- Created reusable UVM components (driver, monitor, scoreboard, sequences) to drive simulations with randomized and directed stimuli, achieving exhaustive functional coverage through FSM verification and assertion-based checking.

VHDL 4-Bit Calculator with Testbenching and FSM Design 🔗 | VHDL, Testbenching, FPGA, RTL

- Produced and implemented a 4-bit calculator on FPGA using VHDL, supporting addition and accumulator clear (AC) operations with finite state machine (FSM) control logic.
- Architected modular RTL components including datapath (adder, 8-bit padding), FSM control circuit, and a register module interfaced via pushbuttons and slide switches.
- Created a comprehensive VHDL testbench to verify functional correctness of “Add” and “AC” operations, including state transitions and output waveform validation, utilized from Xilinx Vivado for simulation and synthesis
- Mapped onto a FPGA board with constraints file to interface slide switches, LEDs, pushbuttons, and a 100MHz clock.

Technical Skills

Technical: SystemVerilog, UVM, Verilog, VHDL, Questa, Vivado, Coverage-driven Verification, Constrained-random Verification, Assertion-based Verification, RTL Verification, SoC Verification, FPGA, ASIC, Python, C++, Linux, Assembly